

Beacon Community Capital — Loan Approval Decision Support (Logistic Regression)

Project overview

Beacon Community Capital (a community development lender in Columbus, Ohio) needs to handle a growing volume of loan applications while keeping decisions consistent, explainable, and human-accountable. This project builds a **reproducible** machine learning workflow in a Jupyter notebook to **support** (not replace) loan officers by triaging applications into confidence bands.

Business problem

- Application volume increased while staffing stayed flat, slowing decision times.
- Manual reviews can vary between officers, and reasoning is not always consistently documented.
- A single hard FICO cutoff (e.g., 660) is too simplistic for Beacon's mission and borrower profile.

Project objective

- Train a **logistic regression classifier** to predict Beacon's **historical loan officer approval decision** (approved), not

borrower repayment/default risk.

- Produce **approval probabilities** (0–1) and apply business-defined triage thresholds:
 - $p \geq 0.80 \rightarrow$ **Expedited** workflow (likely approval)
 - $0.20 \leq p < 0.80 \rightarrow$ **Send to human** (mandatory officer review)
 - $p < 0.20 \rightarrow$ **Likely rejection, still requires human review**

Dataset

- File: `loan_applications.csv`
- Each row: one historical loan application
- Key columns:
 - Identifier: `applicant_id` (never used as a model feature)
 - Inputs: `fico_score`, `annual_income`, `loan_amount`, `loan_term_months`, `employment_status`, `years_employed`, `savings_balance`
 - Target: `approved` (historical decision)

Notebook stages (4 broad steps)

1. Load & clean the data
2. Explore historical approvals and applicant characteristics (EDA)
3. Train a logistic regression model
4. Evaluate model performance and interpret results

Setup / dependencies

This project uses Python with common data science libraries:

- `pandas`, `numpy`

- `matplotlib`, `seaborn`
- `scikit-learn`

Environment (recommended)

- Python 3.10+ (3.9+ typically works)
- VS Code with the Jupyter extension

How to run

1. Open `Learner_Guide_Notebook.ipynb` in VS Code.
2. Select a Python kernel with the dependencies installed.
3. Run cells top-to-bottom to reproduce the full workflow.

Data cleaning summary (performed in-notebook)

The notebook documents each decision and prints counts before/after each step:

- Converted money columns from text like "`$53,000`" to numeric:
 - `annual_income`, `loan_amount`, `savings_balance`
 - Removed exact duplicate rows to prevent bias.
 - Removed rows with impossible values due to likely data-entry errors (without removing blank FICO rows), including:
 - `fico_score > 850` (when not missing)
 - `annual_income < 0`
 - `loan_amount <= 0`
 - `loan_term_months > 84`
 - `years_employed > 60`
 - `savings_balance < 0`
 - Filled missing `fico_score` values using the **median** FICO score.
-

Exploratory data analysis (EDA)

The notebook includes:

- Approved vs rejected counts + bar chart
- FICO histogram + a vertical cutoff line at 660
- Approval rate by `employment_status` + bar chart
- Box plot of FICO by approval decision (to show overlap)
- Correlation heatmap across numeric features and approval outcome

Modeling approach

- Target engineering:
 - `approved_flag` created as 1 for Approved and 0 for Rejected
- Feature set (X) includes only numeric application characteristics plus a simple employment indicator:
 - `fico_score`, `annual_income`, `loan_amount`, `loan_term_months`, `years_employed`, `savings_balance`
 - `is_self_employed = 1` if `employment_status == "Self-employed"` else 0
- Leakage prevention:
 - Excludes `applicant_id`, `approved`, and `approved_flag` from features
- Train/test split:
 - `test_size=0.2`, `stratify=y`, `random_state=42`
- Model:
 - `LogisticRegression(max_iter=1000)`
- Interpretation:
 - Plots raw model coefficients
 - Fits a **separate** scaled model (via `StandardScaler` fit on `X_train` only) to compare coefficient magnitudes more fairly across different units

Evaluation & key results

Evaluation is performed on the **test set only**:

- Test accuracy for the logistic regression model: **(see notebook output)**
- Flat rule baseline (approve if `fico_score` ≥ 660) accuracy: **(see notebook output)**
- Always-approve baseline accuracy: **(see notebook output)**

The notebook also uses `predict_proba()` to bucket test applications into:

- “Auto-approve” (`p` > 0.80)
- “Send to human” (`0.20` $\leq p \leq 0.80$)
- “Auto-reject” (`p` < 0.20) — **still requires human review per business policy**

Fairness & risk notes

- Even without race/age, variables like income, savings, and employment can correlate with protected characteristics and may still produce unfair outcomes for some groups.
- Because the model learns **historical officer decisions**, it can reproduce (and potentially amplify) past inconsistency or bias.
- The borderline band intentionally routes cases to humans to reduce harm from uncertain model decisions and preserve accountability.

Limitations

- Predicts historical approvals, not actual repayment/default outcomes.
- Small datasets can be sensitive to noise and may not generalize well to new applicant populations or changing policy.

- Coefficients indicate association with historical decisions, not causality.

Appropriate use

- Use as a **decision-support triage tool** to prioritize officer attention and speed up clearly strong applications.
- Do **not** use as an automatic rejection system; loan officers remain responsible for final decisions.